LOGISTIC REGRESSION AS A NEURAL NETWORK

OUTLINE

- Binary Classification
- Logistic Regression
- Logistic Regression Cost Function
- Gradient Descent
- Derivatives
- Computation Graph
- Derivatives with a Computation Graph
- Logistic Regression Gradient Descent
- Gradient Descent on m examples



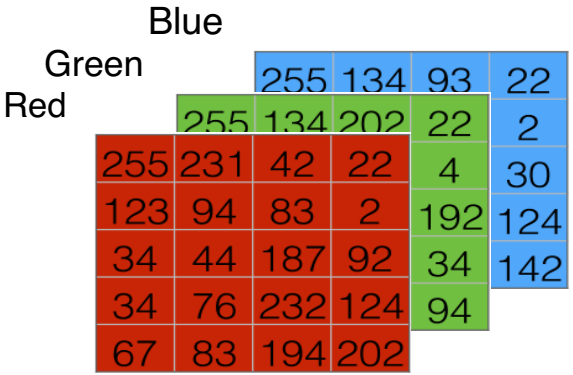
deeplearning.ai

Basics of Neural Network Programming

Binary Classification

Binary Classification





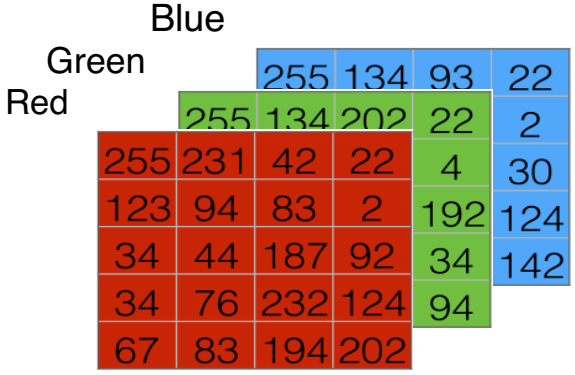
1 (cat) vs 0 (non cat)

Binary Classification



y: output, label

1 (cat) vs 0 (non cat)



- Image size: 64 x 64 pixels
- 3 color channels: R, G, B
- Pixel intensities $\in [0, 255]$ (8 bits per color)

Convert pixel intensities into a feature vector x:

$$x = \begin{bmatrix} R \\ G \\ B \end{bmatrix} = \begin{bmatrix} 255 \\ 231 \\ \vdots \\ 255 \\ 134 \\ \vdots \\ 255 \\ 134 \\ \vdots \end{bmatrix} \begin{matrix} \} R: 64 \times 64 \\ \\ \\ \} G: 64 \times 64 \\ \\ \\ \} B: 64 \times 64 \end{matrix} \quad 64 \times 64 \times 3 = 12\,288$$

$n = n_x = 12288$ → Dimension of input vector

Goal of Binary classification:

Given input feature vector x estimate the corresponding label y: $x \longrightarrow y$

Notation

- A single training sample (example)

  (x, y) , $x \in \mathbb{R}^{n_x}$, $y \in \{0, 1\}$

  x: feature vector

  y: label

- Training set: m training examples

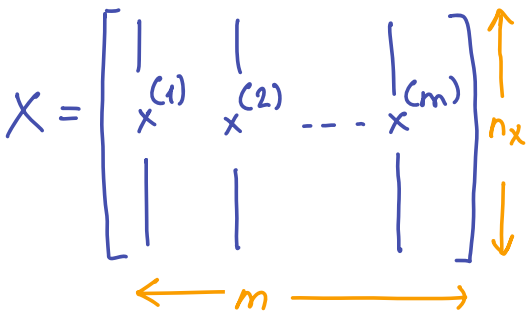
  $(x^{(1)}, y^{(1)})$: input and output for the first training example

  $\{(x^{(1)}, y^{(1)}), (x^{(2)}, y^{(2)}), \ldots, (x^{(m)}, y^{(m)})\}$

  $m = m_{train}$: # training examples

  m_{test} : # test examples

- Represent the training set features with a matrix:



$X \in \mathbb{R}^{n_x \times m}$ in Python: X.shape = (n_x, m)

- Represent the labels of the training set with a vector:

$$Y = [y^{(1)}, y^{(2)}, \dots, y^{(m)}]$$

$Y \in \mathbb{R}^{m}$, in Python Y.shape = $(1, m)$



deeplearning.ai

Basics of Neural Network Programming

Logistic Regression

Logistic Regression

Goal: Given a picture represented by a feature vector x, we want to estimate the probability that this is a cat picture.

Given x, want $\hat{y} = P(y=1|x)$, $\hat{y}$: estimated label

$x \in \mathbb{R}^{n_x}$, $0 \leq \hat{y} \leq 1$

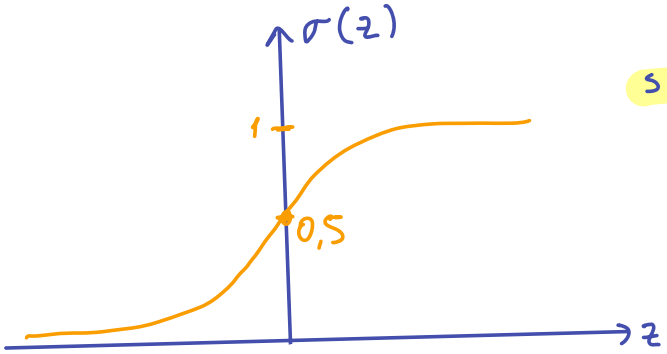
Parameters of logistic regression:

$w \in \mathbb{R}^{n_x}$, $b \in \mathbb{R}$

Output : $\hat{y} = \underbrace{w^T x + b}$ (a linear function of the input x)

not btwn. 0 and 1

$$\hat{y} = \sigma(\underbrace{w^T x + b}_{z})$$



sigmoid or logistic function

$$\sigma(z) = \frac{1}{1+e^{-z}}$$

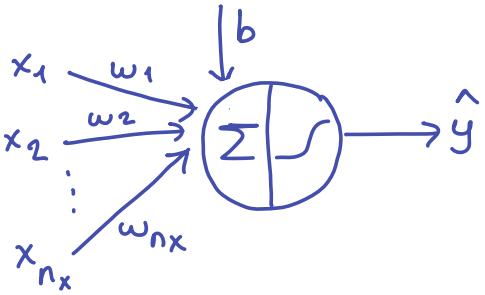
if z is large, $\sigma(z) \approx \frac{1}{1+0} \cong 1$

if z is a large negative number,

$$\sigma(z) = \frac{1}{1+e^{-z}} \approx \frac{1}{1+\text{Bignumber}} \approx 0$$

$$\hat{y} = \sigma\left(\sum_{i=1}^{nx} w_i x_i + b\right)$$

Implemented using a single neuron:



Alternative notation:

Define an extra feature: $x_0 = 1$

$$x = \begin{bmatrix} x_0 \\ x_1 \\ \vdots \\ x_{n_x} \end{bmatrix} \Rightarrow x \in \mathbb{R}^{n_x+1}$$

$$\hat{y} = \sigma(\theta^T x)$$

$$\theta = \begin{bmatrix} \theta_0 \\ \theta_1 \\ \vdots \\ \theta_{n_x} \end{bmatrix} \begin{matrix} \} b \\ \} \omega \end{matrix}$$

Question

What are the parameters of logistic regression?

- ○ W and b, both n_x dimensional vectors.
- ○ W, an identity vector, and b, a real number.
- ○ W and b, both real numbers.
- ○ W, an n_x dimensional vector, and b, a real number.



deeplearning.ai

Basics of Neural Network Programming

Logistic Regression cost function

Logistic Regression cost function

$\hat{y} = \sigma(w^T x + b)$, where $\sigma(z) = \frac{1}{1+e^{-z}}$

Goal: Learn w and b

Given $\{(x^{(1)}, y^{(1)}), \ldots, (x^{(m)}, y^{(m)})\}$, want $\hat{y}^{(i)} \approx y^{(i)}$.

Loss (error) function:

$\hat{y}^{(i)} = \sigma(w^T x^{(i)} + b)$

$z^{(i)} = w^T x^{(i)} + b$

$\sigma(z^{(i)}) = \sigma(w^T x^{(i)} + b)$

superscript denotes $(i)^{th}$ example

- The loss function measures the discrepancy between the prediction $\hat{y}^{(i)}$ and the desired output $y^{(i)}$.

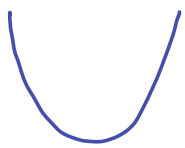
Andrew Ng

- Loss function is the error for a single training example.

e.g. $\mathcal{L}(\hat{y}, y) = \frac{1}{2}(\hat{y} - y)^2$

- But this function is not used for logistic regression since the optimization problem becomes non-convex, i.e. there are multiple local optima.

convex, single min





- non-convex,
- multiple local min.
- gradient descent may not work well.

Loss Function for Logistic Regression: Binary Cross-Entropy

$$\mathcal{L}(\hat{y}, y) = -y \log \hat{y} - (1-y) \log(1-\hat{y})$$

- We want the loss to be as small as possible.

Intuition:

→ If $y = 1$: $\mathcal{L}(\hat{y}, y) = -\log \hat{y}$: want to minimize this

want $\log \hat{y}$ to be as large as possible

remember: $0 \leq \hat{y} \leq 1$

⇒ $\hat{y}$ should be close to 1 (true label)

⇒ $\log \hat{y}$ should be close to 0 (zero loss)

(if $\hat{y} \approx 0$ ⇒ $-\log \hat{y}$ is very large)

→ If $y = 0$: $\mathcal{L}(\hat{y}, y) = -\log(1-\hat{y})$

want this as large as possible

⇒ $\hat{y}$ should be close to 0.

$$\Rightarrow \mathcal{L}(\hat{y}, y) \approx 0.$$

(If $\hat{y} \approx 1 \Rightarrow -\log(0) = \infty$)

Cost Function (Cost of the parameters)

- Measures how well the classifier is doing on the whole training set.
- Average of the loss function on the entire training set.
- We want to find the parameters that minimize the overall cost function.

$$J(w,b) = \frac{1}{m} \sum_{i=1}^{m} \mathcal{L}(\hat{y}^{(i)}, y^{(i)})$$

$$J(w,b) = \frac{-1}{m} \sum_{i=1}^{m} \left[y^{(i)} \log \hat{y}^{(i)} + (1-y^{(i)}) \log (1-\hat{y}^{(i)}) \right]$$

Question

What is the difference between the cost function and the loss function for logistic regression?

- ◯ The loss function computes the error for a single training example; the cost function is the average of the loss functions of the entire training set.
- ◯ The cost function computes the error for a single training example; the loss function is the average of the cost functions of the entire training set.
- ◯ They are different names for the same function.



deeplearning.ai

Basics of Neural Network Programming

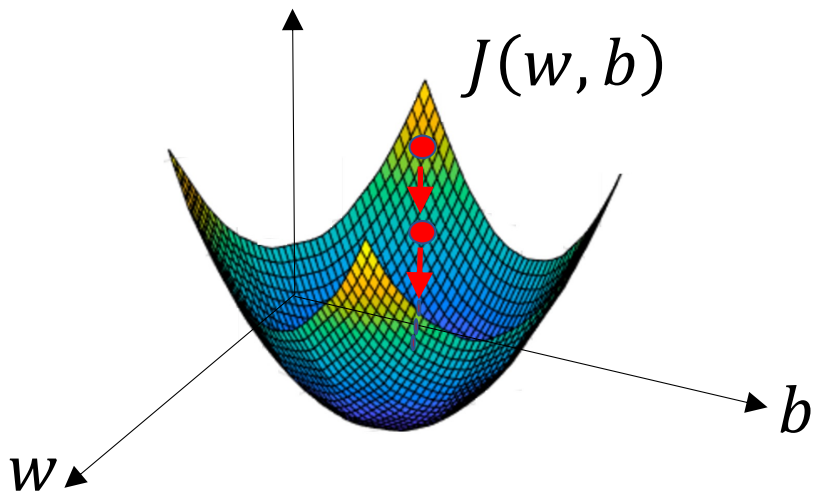
Gradient Descent

Gradient Descent : Logistic Regression

Recap: $\hat{y} = \sigma(w^T x + b)$, $\sigma(z) = \frac{1}{1+e^{-z}}$

$$J(w,b) = \frac{1}{m}\sum_{i=1}^{m} \mathcal{L}(\hat{y}^{(i)}, y^{(i)}) = -\frac{1}{m}\sum_{i=1}^{m} y^{(i)} \log \hat{y}^{(i)} + (1 - y^{(i)}) \log(1 - \hat{y}^{(i)})$$

Want to find w, b that minimize $J(w,b)$



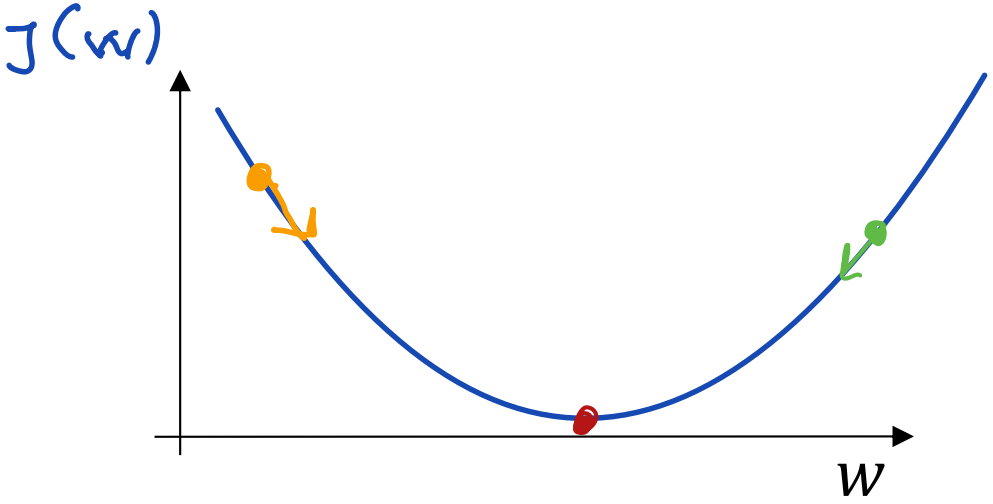
Goal: Learn the parameters w, b that minimize the cost function J(w, b)

Andrew Ng

- Simplified illustration of J(w, b)
- Convex function: has a single global min.

- $J(w, b)$ is a convex function
- To minimize $J(w, b)$
  - initialize w, b randomly
  - apply gradient descent

Gradient Descent



Repeat {

$$w := w - \alpha \frac{dJ(w)}{dw}$$

Learning rate

}

- Learning rate controls the step size at each iteration.

For $J(w,b)$:

$$w := w - \alpha \frac{\partial J(w,b)}{\partial w}$$

$$b := b - \alpha \frac{\partial J(w,b)}{\partial b}$$

∂ : partial derivative

Question

True or false. A convex function always has multiple local optima.

- ○ True
- ○ False

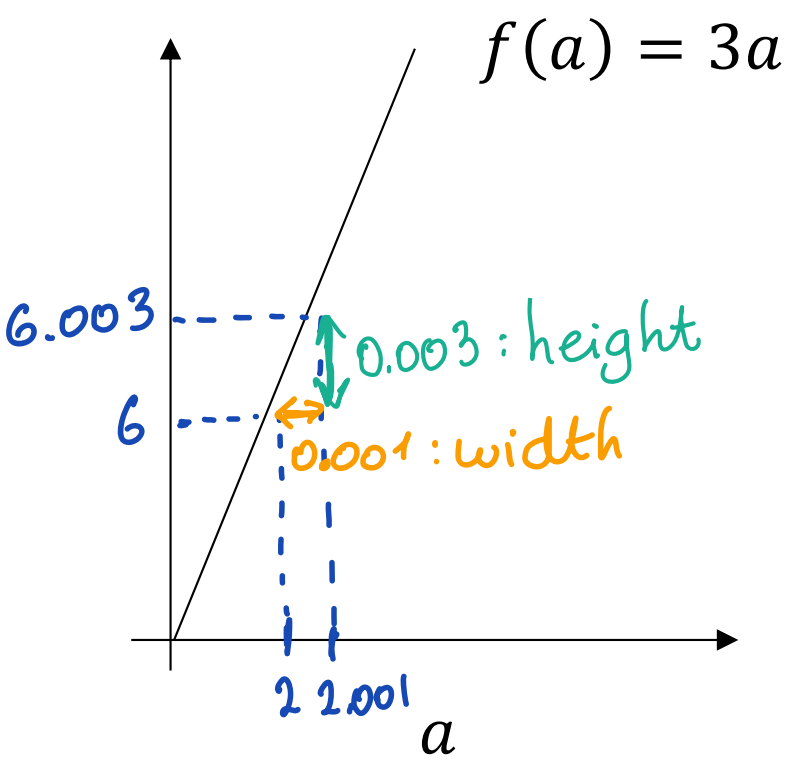


deeplearning.ai

Basics of Neural Network Programming

Derivatives

Intuition about derivatives



- Let $a = 2$ $\quad f(a) = 6$
  $+0.001$ → $a = 2.001$ $\quad f(a) = 6.003$ ($+0.003$)
- Slope (derivative) of $f(a)$ at $a = 2$ is 3.

$$\Delta \frac{\text{height}}{\text{width}} = \frac{0.003}{0.001}$$

- Let $a = 5$ $\quad f(a) = 15$
  $+0.001$ → $a = 5.001$ $\quad f(a) = 15.003$ ($+0.003$)

Slope at $a = 5$ is also 3

$$\frac{df(a)}{da} = 3 = \frac{d}{da} f(a)$$

Andrew Ng

Question

True or false. A convex function always has multiple local optima.

- ◯ True
- ◯ False

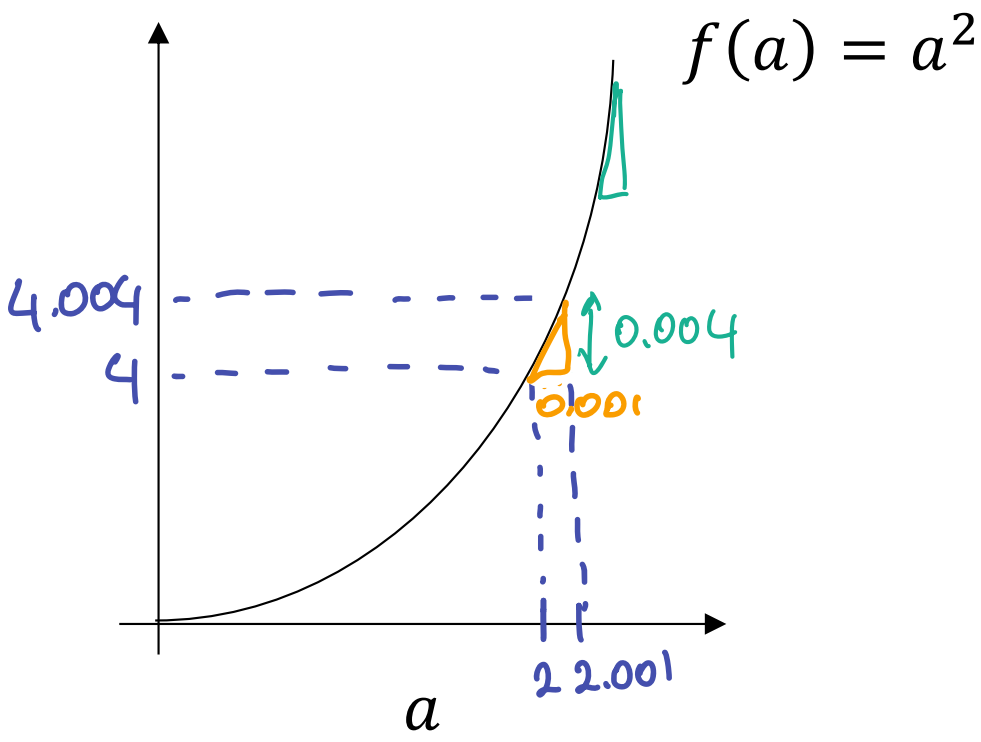


deeplearning.ai

Basics of Neural Network Programming

More derivatives examples

Intuition about derivatives



- Let $a = 2$ $\quad f(a) = 4$ $\xrightarrow{+0.004}$
  $\xrightarrow{+0.001}$ $a = 2.001$ $\quad f(a) \cong 4.004$

- Slope (derivative) of $f(a)$ at $a = 2$ is 4

  $\frac{d}{da} f(a) = 4$ when $a = 2$

- Let $a = 5$ $\quad f(a) = 25$ $\xrightarrow{+0.01}$
  $\xrightarrow{+0.001}$ $a = 5.001$ $\quad f(a) \cong 25.010$

$\frac{d}{da} f(a) = 10$ at $a = 5$

$\frac{d}{da} f(a) = \frac{d}{da} a^2 = 2a$

Andrew Ng

More derivative examples

$f(a) = a^2$

$\frac{d}{da} f(a) = 2a$

$a = 2 \quad f(a) = 4$

$a = 2.001 \quad f(a) \cong 4.004$

$f(a) = a^3$

$\frac{d}{da} f(a) = 3a^2$

$+0.01$ ↓ $a = 2 \quad f(a) = 8$ (0.12)

$a = 2.001 \quad f(a) \cong 8.012$

$3a^2 = 3(2)^2 = 12$

$f(a) = \underbrace{\log_e(a)}_{\ln(a)}$

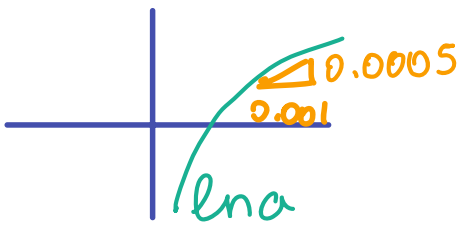
$\frac{d}{da} f(a) = \frac{1}{a}$

$+0.001$ ↓ $a = 2 \quad f(a) \cong 0.69315$

$a = 2.001 \quad f(a) \cong 0.69365$

$+0.0005$

$= \frac{1}{2}$ at $a = 2$





deeplearning.ai

Basics of Neural Network Programming

Computation Graph

Computation Graph

Operation of a Neural network consists of 2 stages:

1) Forward propagation to compute the output

2) Backward propagation to update the weights to minimize the error. (compute derivatives)

Computation graph explain why we have 2 step.

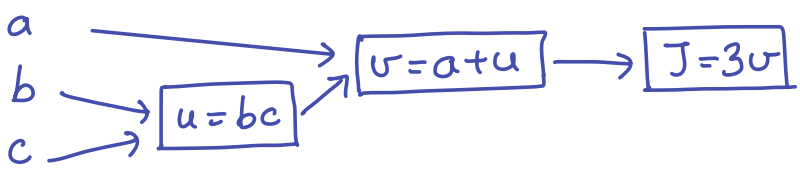
Ex Let us compute the function:

$$J = 3(a + bc)$$

Computation of J consists of 3 steps:

$$u = bc$$
$$v = a + u$$
$$J = 3v$$

There 3 steps can be represented with a computation graph:



eg: if $a = 5, b = 3, c = 2 \Rightarrow u = 6, v = 11, J = 33$

- Computation graph is useful if there's an output (cost) function J, which you want to optimize.
- Using a left-to-right pass (forward propagation) we computed J.
- Using a right-to-left pass (back propagation) to compute derivatives.

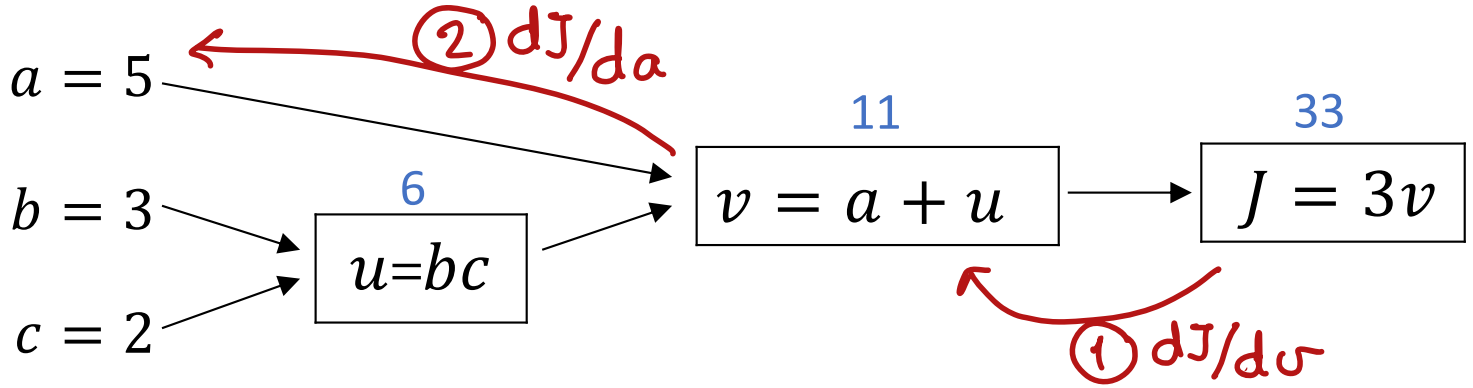


deeplearning.ai

Basics of Neural Network Programming

Derivatives with a Computation Graph

Computing derivatives



① Compute $\frac{dJ}{dv} = ?$ i.e. if we change v a little how would J change?

$J = 3v$

$v = 11$ —(change a little, +0.001)→ 11.001

$J = 33$ —(+0.003, 3 times as much)→ 33.003

$\Rightarrow \frac{dJ}{dv} = \frac{0.003}{0.001} = 3$

Andrew Ng

② Compute $\frac{dJ}{da}$. i.e. if we change a little, how does that effect the value of J = ?

$a = 5 \xrightarrow{+0.001} 5.001$

$v = 11 \xrightarrow{+0.001} 11.001$

$J = 33 \xrightarrow{+0.003} 33.003$

$$\frac{dv}{da} = \frac{0.001}{0.001} = 1$$

$$\frac{dJ}{da} = \frac{0.003}{0.001} = 3$$

If we change a, it will change v, which will change J.

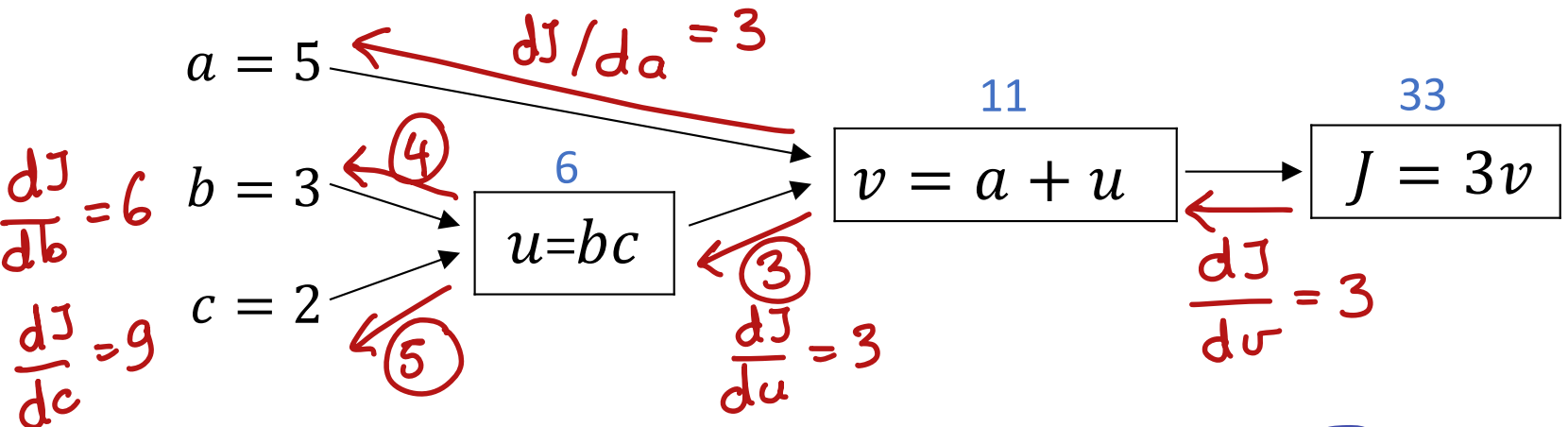
$$\Rightarrow \frac{dJ}{da} = \frac{dJ}{dv} \cdot \frac{dv}{da} \quad \text{(chain rule)}$$

$$= 3 \cdot 1$$

$$= 3$$

$\frac{d\ \text{FinalOutputVar}}{d\ \text{var}}$ ↳ intermediate variable

Computing derivatives



(3) Compute $\frac{dJ}{du}$.

$u = 6 \longrightarrow 6.001$

$v = 11 \longrightarrow 11.001$

$J = 33 \longrightarrow 33.003$

$$\frac{dJ}{du} = \frac{0.003}{0.001} = 3$$

using chain rule : $\frac{dJ}{du} = \underbrace{\frac{dJ}{dv}}_{3} \cdot \underbrace{\frac{dv}{du}}_{1} = 3$

Andrew Ng

④ Compute $\frac{dJ}{db} = \underbrace{\frac{dJ}{dv}\cdot\frac{dv}{du}}_{\frac{dJ}{du}=3} \cdot \underbrace{\frac{du}{db}}_{2} = 6$

$b=3 \longrightarrow 3.001$
$u=bc=6 \longrightarrow 6.002$
$v=11 \longrightarrow 11.002$
$J=33 \longrightarrow 33.006$

$\Rightarrow \frac{du}{db} = \frac{0.002}{0.001} = 2$

$\frac{dJ}{du} = 3$

⑤ Compute $\frac{dJ}{dc} = \underbrace{\frac{dJ}{du}}_{3} \cdot \underbrace{\frac{du}{dc}}_{b=3\ (u=bc)} = 9$

Shorthand notation for implementation

$\frac{dJ}{da}$: "da"

in general dvar : derivative of J w.r.t an intermediate variable.



deeplearning.ai

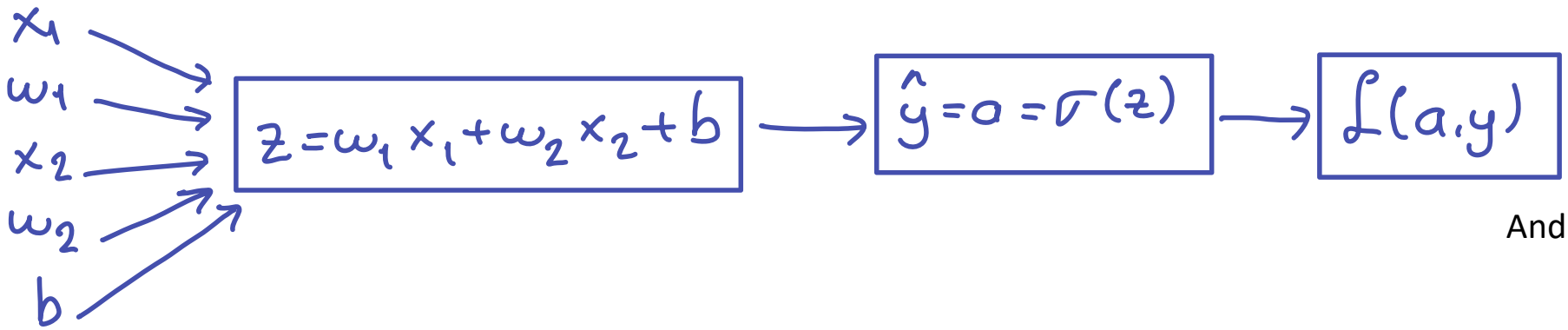
Basics of Neural Network Programming

Logistic Regression Gradient descent

Logistic regression recap

$$z = w^T x + b$$
$$\hat{y} = a = \sigma(z)$$
$$\mathcal{L}(a, y) = -(y \log(a) + (1 - y) \log(1 - a))$$

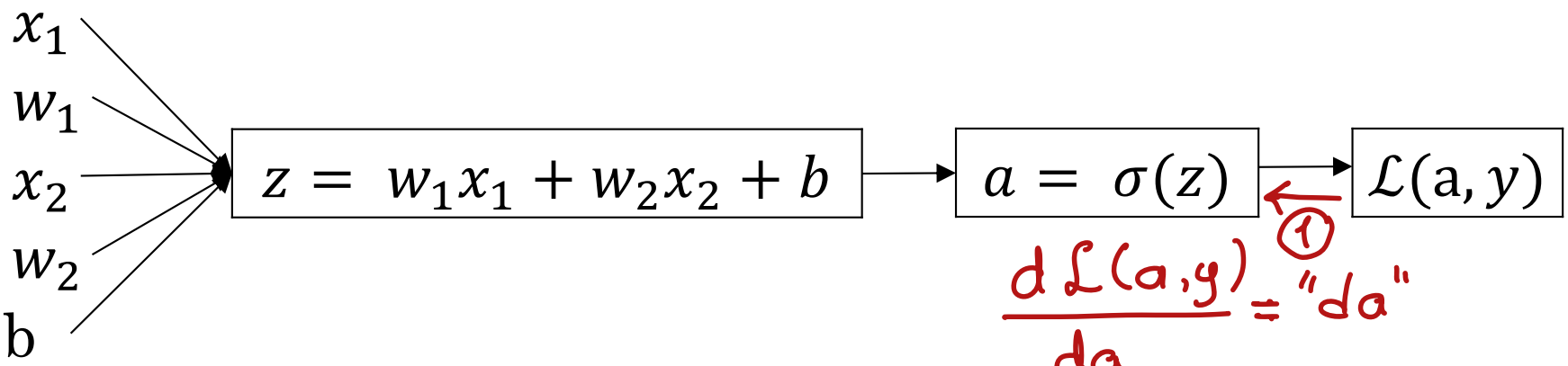
Goal: Implement logistic regression using gradient descent and computation graph.

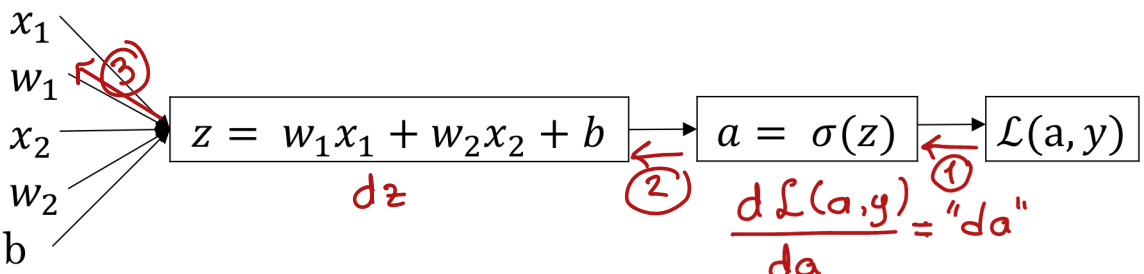


Andrew Ng

* To reduce the loss $\mathcal{L}(a, y)$ modify the parameters w_1, w_2, b.

Logistic regression derivatives





(2) $dz = \frac{d\mathcal{L}}{dz} = \frac{d\mathcal{L}}{da} \cdot \frac{da}{dz} = a - y$

$\frac{d\mathcal{L}}{da} = -\frac{y}{a} + \frac{1-y}{1-a}$, $\frac{da}{dz} = a(1-a)$

(3) $\frac{d\mathcal{L}}{dw_1} = "dw_1" = x_1 dz$

- Similarly $\frac{d\mathcal{L}}{dw_2} = x_2 . dz$, $db = dz$

- Finally, update the parameters

$$w_1 := w_1 - \alpha\, dw_1$$
$$w_2 := w_2 - \alpha\, dw_2$$
$$b := b - \alpha\, db$$



deeplearning.ai

Basics of Neural Network Programming

Gradient descent on m examples

Logistic regression on m examples

loss for 1 sample

The cost function is: $J(w,b) = \frac{1}{m}\sum_{i=1}^{m} \mathcal{L}(a^{(i)}, y^{(i)})$

where $a^{(i)} = \hat{y}^{(i)} = \sigma(z^{(i)}) = \sigma(w^T x^{(i)} + b)$

Previously, given 1 sample $(x^{(i)}, y^{(i)})$, we computed:

$dw_1^{(i)}, dw_2^{(i)}, db^{(i)}$

For m examples:

$$\frac{\partial}{\partial w_1} J(w,b) = \frac{1}{m}\sum_{i=1}^{m} \underbrace{\frac{\partial}{\partial w_1}\mathcal{L}(a^{(i)}, y^{(i)})}_{dw_1^{(i)}}$$

Andrew Ng

Logistic regression on m examples (Pseudo-code)

Initialize: $J=0, dw_1=0, dw_2=0, db=0$

For $i=1$ to m

$z^{(i)} = w^T x^{(i)} + b$

$a^{(i)} = \sigma(z^{(i)})$

$J \mathrel{+}= -\left[y^{(i)} \log a^{(i)} + (1-y^{(i)}) \log(1-a^{(i)})\right]$

$dz^{(i)} = a^{(i)} - y^{(i)}$

$dw_1 \mathrel{+}= x_1^{(i)} dz^{(i)}$

$dw_2 \mathrel{+}= x_2^{(i)} dz^{(i)}$

$db \mathrel{+}= dz^{(i)}$

} if $n=2$

$J /= m$

$dw_1 /= m$; $dw_2 /= m$; $db /= m$

} Note: $dw_1 = \frac{dJ}{dw_1}$

Andrew Ng

$w_1 += w_1 - \alpha dw_1$

$w_2 += w_2 - \alpha dw_2$

$b += b - \alpha db$

- The above is 1 step of gradient descent.
- Run multiple steps until convergence.

- The above code needs 2 for loops:
  - For the sampler (1 to m)
  - For the parameters (1 to n)
- Using vectorization we can get rid of the for loops. Code becomes efficient.

Question

In the for loop depicted in the video, why is there only one *dw* variable (i.e. no i superscripts in the for loop)?

- ◯ The value of *dw* in the code is cumulative.
- ◯ Only the derivative of one value is relevant.
- ◯ Only one derivative is being computed.